

PPE COMPLIANCE MONITORING SYSTEM

Using MobileNetV2 Transfer Learning

Thrive Plus Advanced ML & AI · Module 3: Computer Vision & CNNs
Group 5 · Capstone Project · 2026

Group Members

Martha Afful

Michael Douglas Martey

Regina Robertson (Group Leader)

Sandra Ohenewaa Djan

Edward Asare Ansa

Amoah Akolbila Yakubu

Yusif Mohammed Yakubu

Seth Dampson

TABLE OF CONTENTS

1. README	3
Abstract	3
Introduction	3
Problem Statement	3
Objectives	4
Main Results	4
User Interface Overview	4
Running the Application	5
2. DATA CARD	6
Table 2: Dataset Information	6
Dataset Split	6
Image Characteristics	7
Preprocessing	7
Table 5: Data Augmentation Parameters	7
Handling Class Imbalance	8
3. EXPERIMENT LOG	9
What Did Not Work / Was Not Pursued	10
4. MODEL CARD	11
4A: Table 7: Custom CNN from Scratch	11
4B: Table 8: MobileNetV2 Transfer Learning (Production Model)	11
5. RESULTS AND METRICS	13
Models Comparison	13
Confusion Matrices (Three Scratch CNN-Based Models)	13
Table 9: Classification Report Comparison (All Models)	15

MobileNetV2 Results	16
LEARNING CURVES	17
Receiver Operating Characteristic (ROC) Curve and Area Under the Curve (AUC)	
Analysis.....	19
Precision-Recall Analysis	20
Error Analysis (Misclassified Examples)	21
Table 11: Performance Summary	23
6. HOW TO REPRODUCE.....	24
Table 12: Environment	24
Step-by-Step Instructions	24
Step 1: Download and Organise the Dataset.....	24
Step 2: Install Dependencies	24
Step 3: Run the Training Notebook	25
Step 4: Launch the Streamlit Application.....	25
Step 5: Reproduce the Reported Metrics	26
Conclusion, Limitations and Recommendations	26
Appendix: Project File Reference	28
REFERENCES	29

1. README

Abstract

This project presents a deep learning-based system for detecting whether construction workers are wearing safety helmets, aimed at improving workplace safety compliance. Multiple convolutional neural network (CNN) approaches were explored, including a baseline CNN trained from scratch, class imbalance handling techniques (class weighting and oversampling with augmentation), and a transfer learning approach using MobileNetV2 pretrained on ImageNet. Models were evaluated using accuracy, confusion matrices, ROC-AUC scores, precision-recall curves and error analysis on misclassified examples. The final MobileNetV2 model achieved a test accuracy of 98.5% and an AUC of 0.9701, significantly reducing safety-critical misclassifications. The system was deployed as a user-friendly web application using Streamlit, enabling real-time image-based compliance detection.

Introduction

This PPE Compliance Monitoring System is a computer vision application developed to automatically determine whether a construction worker is wearing a safety helmet. The system uses Convolutional Neural Networks (CNNs) and transfer learning with MobileNetV2 to classify images as either Compliant (Helmet) or Non-Compliant (No Helmet).

Problem Statement

The construction and mining industries experience frequent workplace accidents and injuries due to workers failing to wear required Personal Protective Equipment (PPE), particularly safety helmets. Current monitoring methods rely heavily on manual inspections conducted by safety officers, which are time-consuming, inconsistent and unable to provide continuous surveillance across large worksites. As a result, PPE violations may go undetected, increasing the risk of accidents and injuries.

There is therefore a need for an automated, accurate and real-time system that can identify whether workers are complying with helmet safety requirements. This project addresses this problem by developing a computer vision-based PPE Compliance Monitoring System that automatically classifies workers as compliant or non-compliant using image data and deep learning techniques.

Objectives

1. To develop a binary image classification model for helmet compliance detection.
2. To compare a CNN trained from scratch against a transfer learning approach using MobileNetV2.
3. To deploy the best-performing model in a user-friendly interface for safety officers and site managers.
4. To evaluate model performance using standard classification metrics including accuracy, AUC and precision-recall scores.

Main Results

Table 1: Summary of test accuracy achieved by each model variant explored in this project.

Model	Test Accuracy
CNN from Scratch (Baseline)	90.3%
CNN (Class Weights + Augmentation)	88.06%
CNN (Oversampling + Augmentation)	92.54%
MobileNetV2 Transfer Learning ★ PRODUCTION	98.5%

Source: Authors' construct based on data from Kaggle.

From the table 1 above, MobileNetV2 significantly outperforms both CNN variants, achieving 98.5% test accuracy, 8.2 percentage points higher than the baseline CNN (90.3%) and 6.0 percentage points higher than the CNN (Oversampling + Augmentation) (92.54%). For this PPE helmet detection task, transfer learning with MobileNetV2 is the clear production choice over training a CNN from scratch.

User Interface Overview

The PPE Compliance Monitor was deployed using Streamlit to provide a simple and user-friendly interface for safety officers and site managers. The application consists of four pages accessible via a left-hand navigation sidebar:

- Home: project overview, key features and model quick-stats (98.5% accuracy, MobileNetV2, Binary Classification)

- Prediction: image upload (.jpg / .jpeg / .png), an adjustable decision threshold slider (default 0.5), real-time compliance verdict with confidence score, and colour-coded green/red status card
- History: the last 10 predictions stored with timestamp, filename, label, and confidence score
- About: model details, task description, and project goal statement

Running the Application

1. Install all project dependencies (see Section 6 for the exact commands).
2. Launch the Streamlit application: `streamlit run PPE_appNewFinal.py`
3. Navigate to the Prediction page using the left sidebar.
4. Upload a .jpg, .jpeg or .png image of a worker.
5. The model analyses the image and returns a compliance prediction with a confidence score.
6. View previous predictions in the History section if required.

Example outputs:

- COMPLIANT — Helmet Detected (e.g. 98.5% Confidence)
- NON-COMPLIANT — Helmet Not Detected (e.g. 96.2% Confidence)

This section ties nicely to the screenshot of your interface and demonstrates that the deployment requirement of the project was successfully completed.

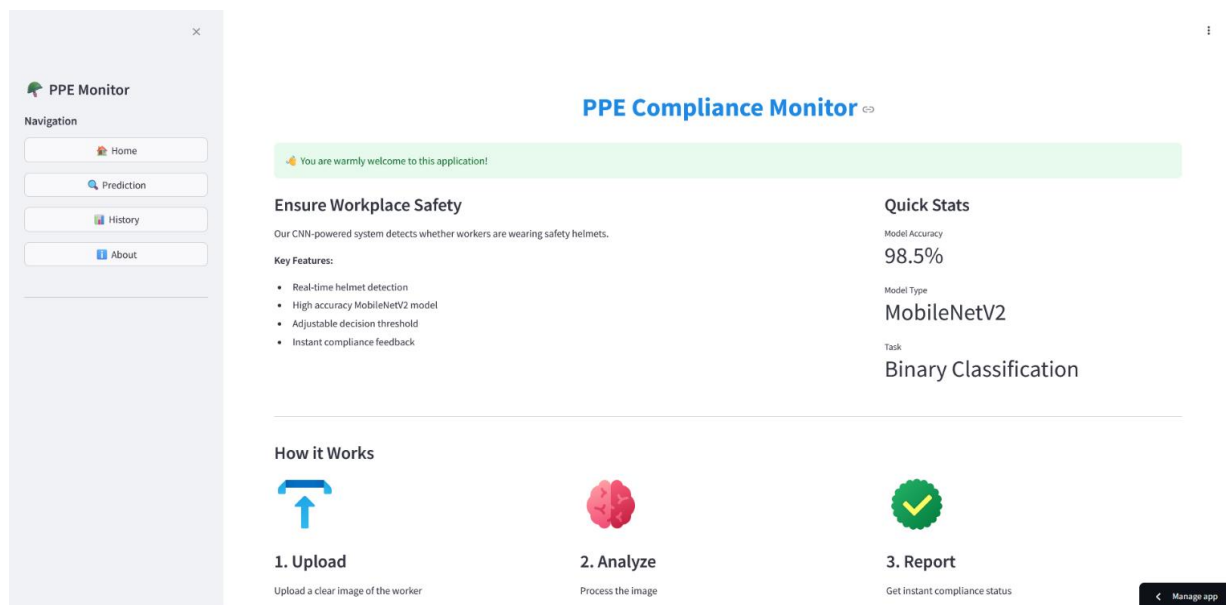


Figure1: A screenshot of PPE Compliance Monitor Streamlit Application

2. DATA CARD

Table 2: Dataset Information

Property	Detail
Dataset Name	PPE Kit Detection (Construction Site Workers)
Source	Kaggle — https://www.kaggle.com/datasets/ketakichalke/ppe-kit-detection-construction-site-workers
Annotation Format	YOLO (.txt files, one per image)
Classification Task	Binary Image Classification
Classes Used	Class 0 = helmet (Compliant), Class 7 = no_helmet (Non-Compliant)
Total YOLO Classes	11 (helmet, gloves, vest, boots, goggles, none, person, no_helmet, no_goggles, no_gloves, no_boots)
Label Logic	If class 0 present: label=1 (Compliant). Elif class 7 present: label=0 (Non-Compliant). Else: discarded.
Random Seed	SEED = 42 (Python random, NumPy, TensorFlow)

Source: Authors' construct based on data from Kaggle.

Dataset Split

Table 3: Binary class distribution per split after applying the label extraction logic.

Split	Helmet (Compliant)	No Helmet (Non-Compliant)	Total
Training	830	230	1,060
Validation	107	27	134
Test	110	24	134
Total	1,047	281	1,328

Source: Authors' construct based on data from Kaggle.

From the above table, the dataset was pre-split by the original Kaggle provider. No additional splitting was performed by the project team. It can be observed that, the training set is significantly imbalanced: Compliant (Helmet) accounts for 78.3% of training samples and Non-Compliant (No

Helmet) for only 21.7%. This imbalance required explicit mitigation to prevent the model from simply predicting the majority class.

Image Characteristics

Table 4: Describes the visual properties of the dataset images relevant to model performance.

Characteristic	Description
Environment	Outdoor construction sites
Worker Distance	Varying distances from camera (close-up to far-field)
Camera Angles	Multiple perspectives: elevated, ground-level and side views
Lighting Conditions	Variable: direct sunlight, shadows and overcast skies
Helmet Visibility	Partial and full helmet visibility present across images
Input Size (CNN)	Resized to 128×128 pixels
Input Size (MobileNetV2)	Resized to 224×224 pixels

Source: Authors' construct based on data from Kaggle.

Preprocessing

The following preprocessing steps were applied before model training:

- Image resizing: all images resized to 128×128 px for the CNN (Oversampling); resized again to 224×224 px for MobileNetV2 using `skimage.transform.resize`
- Pixel normalisation: pixel values divided by 255.0 to scale to $[0, 1]$ for the CNN (Oversampling)
- MobileNetV2 preprocessing: `tensorflow.keras.applications.mobilenet_v2.preprocess_input` applied to rescaled images (scales pixel values to $[-1, 1]$ as expected by the architecture)
- Global random seed SEED=42 set across Python, NumPy and TensorFlow for reproducibility

Table 5: Data Augmentation Parameters

Augmentation Parameter	Value
Rotation range	15° (oversampling run); 20° (class weights run)
Zoom range	0.1 (oversampling); 0.2 (class weights)
Width / Height shift	0.1 (both runs)
Horizontal flip	True
Brightness range	[0.8, 1.2] (class weights run only)
Rescaling	Handled by normalisation step (not inside ImageDataGenerator for oversampling run)

Source: Authors' construct based on data from Kaggle.

Handling Class Imbalance

Three strategies were applied sequentially to address the class imbalance in the training data shown in table 3:

- **Strategy 1:** Class Weights: the loss function was adjusted to penalise misclassification of the minority class more heavily. Weights were manually tuned to {0: 1.2, 1: 1.0} after an initial `compute_class_weight` call. The minority class (No Helmet) receives 1.2× the loss weight of the majority class
- **Strategy 2:** Oversampling: minority class samples were duplicated using `np.repeat` to match the majority class count. `oversample_factor = len(x_helmet) // len(x_no_helmet)`. The training set was then shuffled with `np.random.permutation` before feeding to the model. This creates exact pixel duplicates, not synthetically varied samples
- **Strategy 3:** Data Augmentation was applied on top of oversampling using `ImageDataGenerator` with rotation, zoom, shift and horizontal flip to reduce overfitting to duplicate samples.

3. EXPERIMENT LOG

Table 6 documents every meaningful training run in this project, capturing what changed, why and what the outcome was. All runs used binary cross-entropy loss and the Adam optimiser unless noted otherwise.

Run	Configuration	Rationale	Test Accuracy	Key Metric	Decision
1	Custom CNNAdam (lr=0.001) Epochs: 15 BS: 32 Input: 128×128 No imbalance handling	Establish baseline on raw imbalanced training data	90.3%	AUC ≈ 0.964	Kept as baseline reference
2	Same CNN+ Class Weights {0:1.2, 1:1.0}+ Augmentation (rot 20°, zoom 0.2) Threshold: 0.6	Penalise minority class misclassification without altering data distribution	80.1%	Improved minority recall	Kept for comparison
3	Same CNN+ Oversampling (np.repeat)+ Augmentation (rot 15°, zoom 0.1) No class weights	Balance training data explicitly; augmentation to prevent fitting to duplicates	92.5%	AUC = 0.9750 AP ≈ 1.00	Selected
4	MobileNetV2 (frozen base) ImageNet weights GAP + BN + Dense(128) + Dropout(0.3) lr=1e-4 Epochs: 10 BS: 32 Input: 224×224	Leverage ImageNet pretrained features for fast convergence on a small dataset	97.8%	Strong validation accuracy from epoch 1	Kept: solid frozen baseline
5	MobileNetV2 (fine-tuned) Last 20 base layers unfrozen lr=1e-5 Epochs: 5 BS: 32	Adapt low-level features to the PPE domain for final accuracy boost	98.5%	AUC = 0.9875 AP ≈ 1.00	SELECTED (production model)

Source: Authors' construct based on data from Kaggle.

What Did Not Work / Was Not Pursued

- VGG16 and ResNet50 were imported at the top of the notebook (from `tensorflow.keras.applications`) but were never trained. The team chose MobileNetV2 for its faster inference, smaller parameter count, and suitability for deployment on resource-constrained environments.
- Higher Dropout (0.5) was used in the scratch CNN. This was reduced to 0.3 in the MobileNetV2 head, reflecting the regularisation already provided by the frozen pretrained base and batch normalisation.
- The threshold of 0.6 trialled in Run 2 improved minority class precision at the cost of some recall. The production app defaults to 0.5 with a Streamlit slider for runtime adjustment by the operator.
- Synthetic oversampling techniques such as SMOTE were not used. The team used `np.repeat` for simplicity, which creates exact pixel duplicates rather than genuinely varied synthetic samples. This is a known limitation documented in the results section.

4. MODEL CARD

4A: Table 7: Custom CNN from Scratch

Property	Detail
Architecture	Sequential CNN — 2 convolutional blocks + dense classifier head
Block 1	Conv2D (32 filters, 3×3, same padding, ReLU) → MaxPooling2D (2×2)
Block 2	Conv2D (64 filters, 3×3, same padding, ReLU) → MaxPooling2D (2×2)
Classifier Head	Flatten → Dense (128, ReLU) → Dropout (0.5) → Dense (1, Sigmoid)
Input Shape	(128, 128, 3) — RGB images normalised to [0, 1]
Output	Scalar probability in [0, 1]. Default threshold 0.5 → 0 = Non-Compliant, 1 = Compliant
Optimiser	Adam (lr = 0.001 default)
Loss Function	Binary Cross-Entropy
Epochs	15 per training run (repeated up to 3 times with different imbalance strategies)
Batch Size	32
Known Limitations	No batch normalisation in convolutional blocks. Susceptible to class imbalance without explicit mitigation. Trained on 128×128 inputs — fine detail (e.g. partially visible or distant helmets) may be lost at this resolution.

Source: Authors' construct based on data from Kaggle.

4B: Table 8: MobileNetV2 Transfer Learning (Production Model)

Property	Detail
Base Architecture	MobileNetV2 pretrained on ImageNet (include_top=False, input_shape=(224,224,3))

Total Base Layers	Approximately 155 layers
Training Phase 1	All base layers frozen. Custom head only. lr=1e-4, 10 epochs, BS=32
Custom Head	GlobalAveragePooling2D → BatchNormalization → Dense(128, ReLU) → Dropout(0.3) → Dense(1, Sigmoid)
Training Phase 2 (Fine-Tune)	Last 20 base layers unfrozen. lr=1e-5, 5 epochs, BS=32
Input Shape	(224, 224, 3) — preprocessed with mobilenet_v2.preprocess_input (scales to [-1, 1])
Output	Scalar probability in [0, 1]. Default threshold 0.5 (adjustable in Streamlit UI)
Saved Format	mobilenet_ppe_model.keras (Keras v3 native format)
Inference Requirements	Resize input to 224×224 and apply preprocess_input before prediction
Final Test Accuracy	98.5%
Final AUC	0.9875
Known Limitations	Binary helmet/no-helmet only — does not detect other PPE items. Requires reasonable image quality and resolution. Near-perfect AP scores may partly reflect oversampling artefacts (np.repeat duplicates) rather than true generalisation on unseen data.

Source: Authors' construct based on data from Kaggle.

5. RESULTS AND METRICS

Models Comparison

This section presents a comprehensive evaluation and comparison of four models developed for PPE helmet compliance detection, namely the baseline Scratch CNN, the Scratch CNN with class weighting, the Scratch CNN with oversampling, and the MobileNetV2 transfer learning model. The performance of these models is assessed using multiple evaluation tools, including confusion matrices and classification reports, learning curves, ROC curves, and precision - recall analysis, as well as qualitative inspection of correctly and incorrectly classified examples. Together, these analyses provide a detailed understanding of each model's learning behavior, classification capability and overall generalization performance in identifying compliant and non-compliant PPE usage.

Confusion Matrices (Three Scratch CNN-Based Models)

Confusion Matrix (Base model)- PPE Helmet Compliance

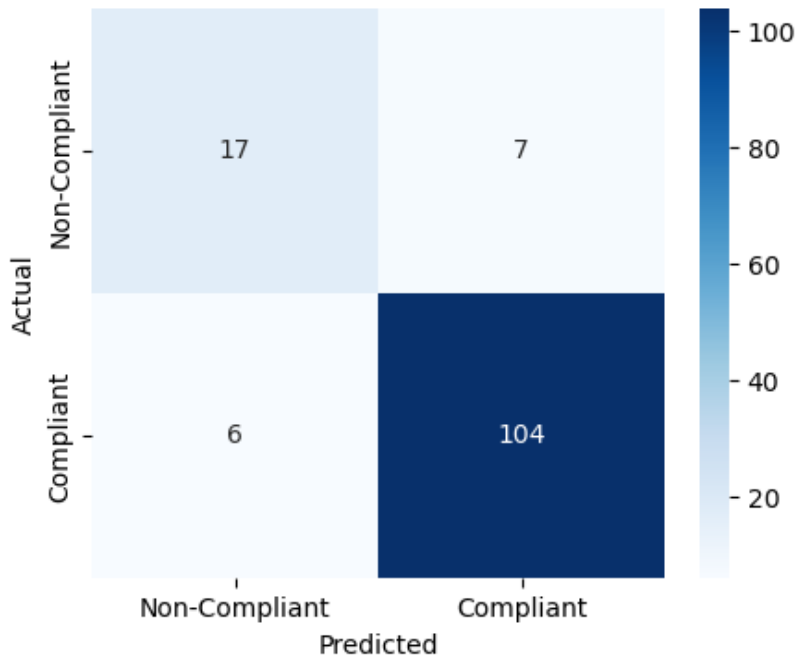


Figure 2: Confusion Matrix (Base model)

Source: Authors' construct based on data from Kaggle.

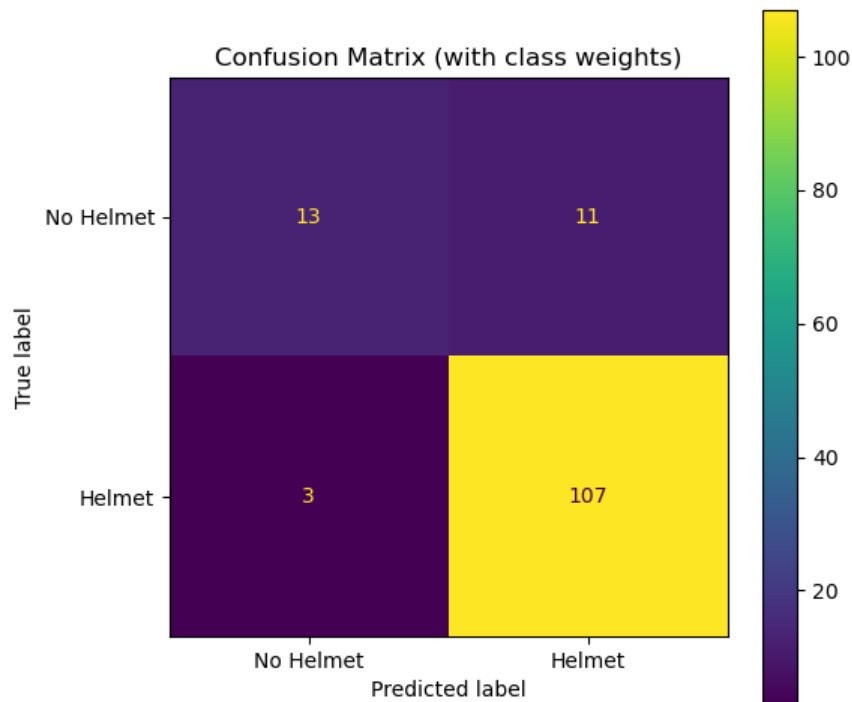


Figure 3: Confusion Matrix (Scratch CNN with Class Weight)

Source: Authors' construct based on data from Kaggle.

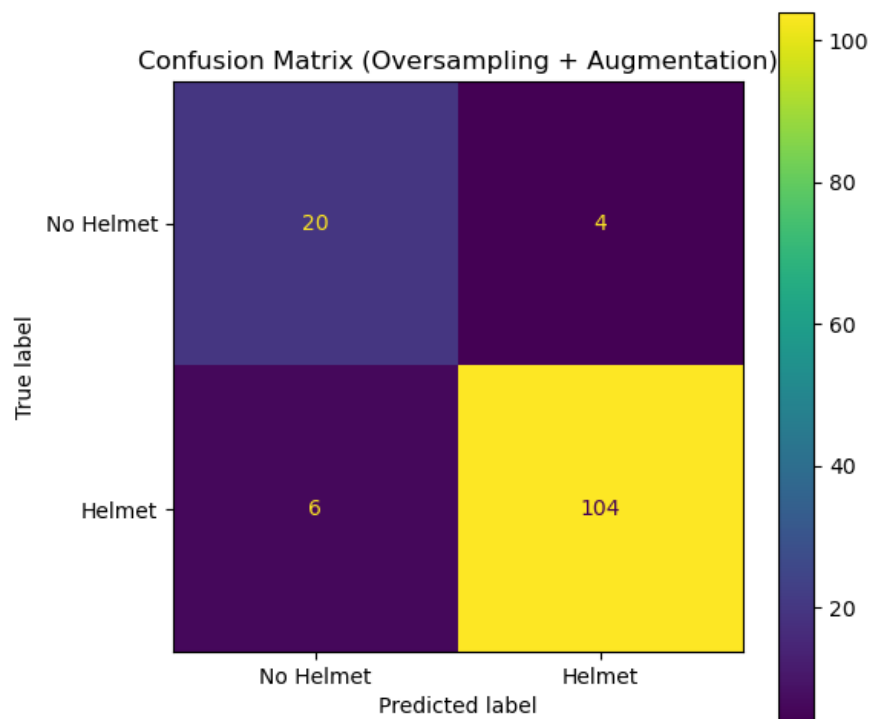


Figure 4: Confusion Matrix (Scratch CNN with Oversampling)

Source: Authors' construct based on data from Kaggle.

Figure 2, 3 and 4 above, presents the confusion matrix results for the three CNN variants. The oversampling-based model achieved the most balanced performance, maintaining a high true positive count (104) while also improving true negatives (20) and minimizing false negatives (4). In contrast, the class-weighted model reduced false positives but introduced more false negatives, indicating a trade-off in classification behavior.

Table 9: Classification Report Comparison (All Models)

Model	Class	Precision	Recall	F1-score	Accuracy	Macro Avg (F1)	Weighted Avg (F1)
Baseline Scratch CNN	Non-Compliant	0.74	0.71	0.72		0.83	
	Compliant	0.94	0.95	0.94	0.90		0.89
Scratch CNN (Class Weights)	Non-Compliant	0.81	0.54	0.65		0.79	
	Compliant	0.91	0.97	0.94	0.90		0.89
Scratch CNN (Oversampling)	Non-Compliant	0.77	0.83	0.80		0.88	
	Compliant	0.96	0.95	0.95	0.93		0.93

Source: Authors' construct based on data from Kaggle.

Table 9 presents a comparative evaluation of the baseline Scratch CNN, class-weighted CNN and oversampling-based CNN. The oversampling approach achieved the best overall performance with an accuracy of 93% and improved macro-F1 score of 0.88, indicating better balance in handling class imbalance compared to the other approaches. Therefore, the Scratch CNN (Oversampling) model was selected as the baseline for the MobileNetV2 transfer learning approach due to its superior overall performance and better balance between precision and recall.

MobileNetV2 Results

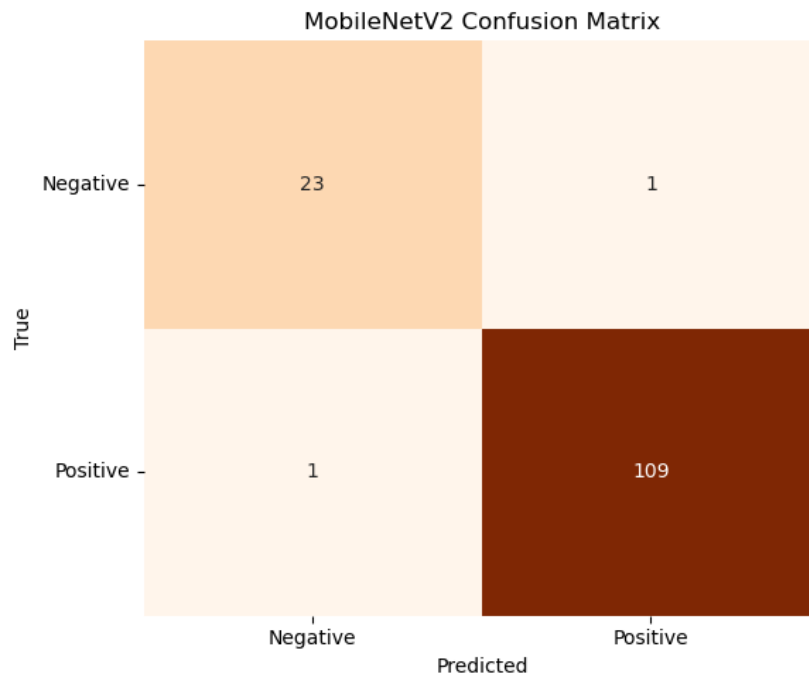


Figure 5: Confusion Matrix (MobileNetV2)

Source: Authors' construct based on data from Kaggle.

TABLE 10: Classification performance of MobileNetV2 transfer learning model for PPE helmet detection using precision, recall, F1-score and support metrics.

Class	Precision	Recall	F1-score	Support
Non-Compliant	0.96	0.96	0.96	24
Compliant	0.99	0.99	0.99	110
Accuracy			0.99	134
Macro Avg	0.97	0.97	0.97	134
Weighted Avg	0.99	0.99	0.99	134

Source: Authors' construct based on data from Kaggle.

Figure 5 and Table 10 present the evaluation of the MobileNetV2 transfer learning model using both the confusion matrix and classification report, and the results show that the model achieves exceptionally high performance in distinguishing between compliant and non-compliant PPE usage. From the confusion matrix in Figure 5, the model correctly identifies 109 true positives and 23 true

negatives, while only producing 1 false positive and 1 false negative, indicating a very low error rate and demonstrating strong sensitivity and specificity in detecting both safety compliance and violations. The classification report in Table 10 further confirms this performance, with precision, recall and F1-scores of 0.99 for the compliant class and 0.96 for the non-compliant class, alongside an overall accuracy of 99% and consistently high macro and weighted averages, showing that the model maintains balanced performance across both classes despite class imbalance.

Overall, the combined results indicate that MobileNetV2 significantly outperforms the Scratch CNN (Oversampling) models, with only two total misclassifications highlighting the effectiveness of transfer learning in learning robust and generalizable features, making the model highly reliable for real-world PPE compliance monitoring systems where both safety detection and minimization of false alarms are critical.

LEARNING CURVES

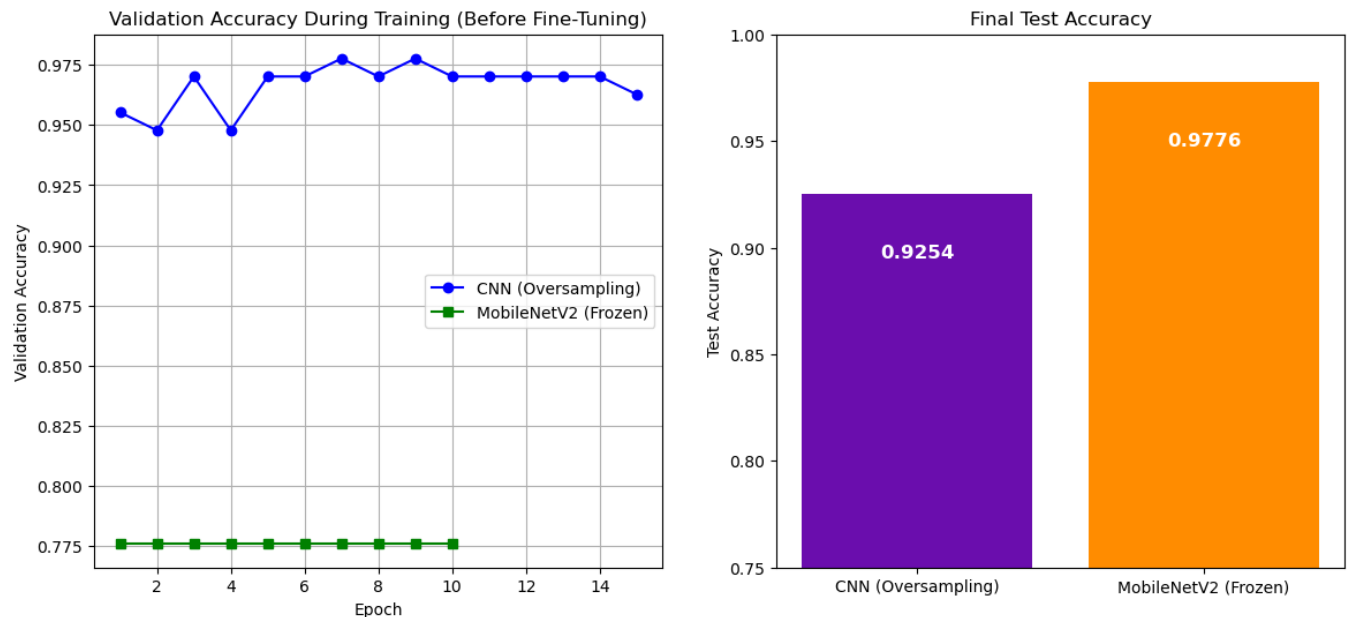


Figure 6: Learning Curve (Before Tuning)

Source: Authors' construct based on data from Kaggle.

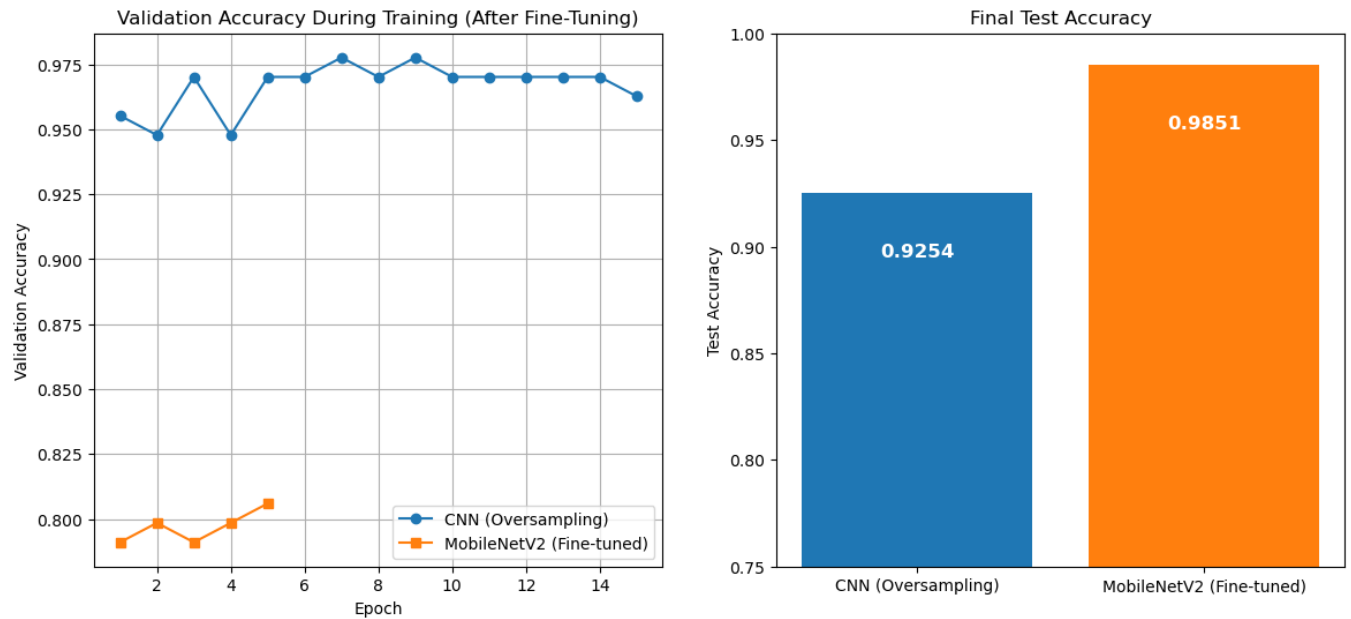


Figure 7: Learning Curve (After Tuning)

Source: Authors' construct based on data from Kaggle.

The MobileNetV2 model initially exhibits limited learning when used as a frozen feature extractor, with validation accuracy plateauing at approximately 0.777 (figure 6), indicating a mismatch between pre-trained features and the PPE classification task. However, upon fine-tuning (figure 7), the model demonstrates a steady improvement in validation performance, highlighting the importance of adapting high-level representations to domain-specific data. Although MobileNetV2 shows slower convergence and lower intermediate validation accuracy compared to the Scratch CNN, its superior final test performance indicates stronger generalization and reduced overfitting. In contrast, the Scratch CNN (Oversampling) achieves higher early validation accuracy but exhibits fluctuations, suggesting instability and sensitivity to the training data. The jump from 97.76% (frozen) to 98.51% (fine-tuned) on the test set further confirms that allowing the pre-trained weights to adapt was highly beneficial. Overall, the transition of MobileNetV2 from underfitting to stable, progressive learning after fine-tuning underscores its effectiveness in achieving robust and transferable feature representations.

Receiver Operating Characteristic (ROC) Curve and Area Under the Curve (AUC) Analysis

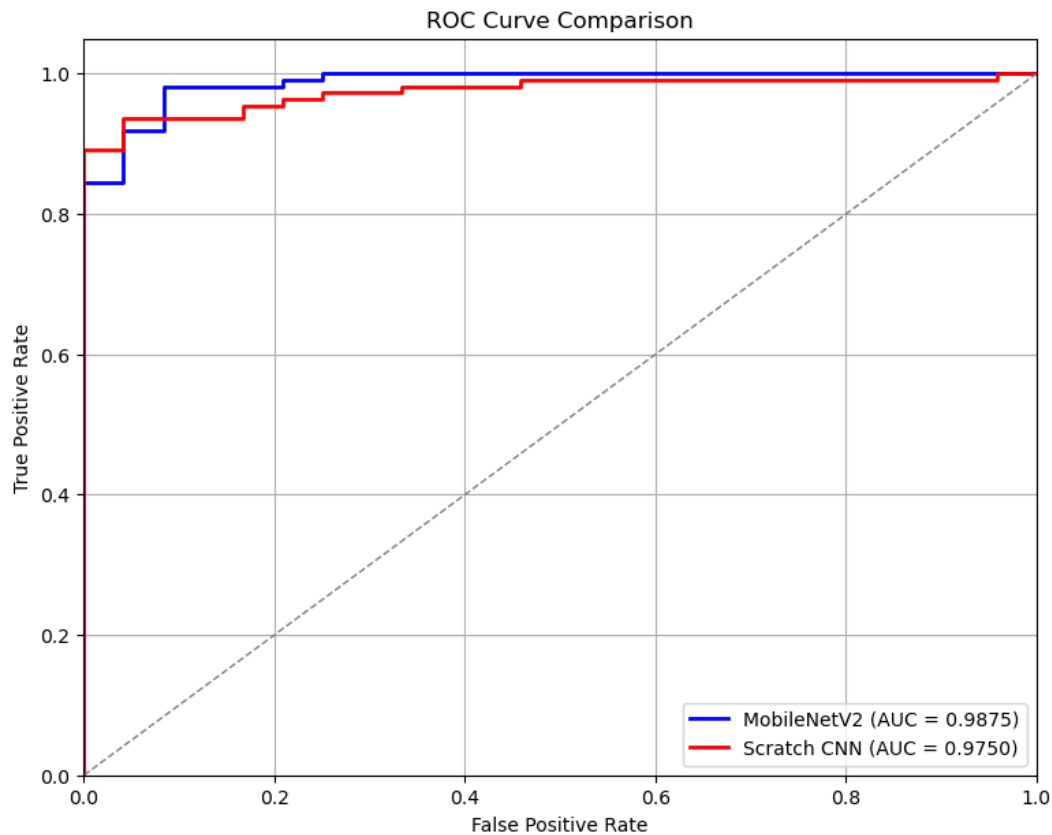


Figure 8: ROC curves and AUC Scores

Source: Authors' construct based on data from Kaggle.

From the plot above, both the Scratch CNN and MobileNetV2 exhibit excellent discriminative performance between Compliant and Non-Compliant classes, with ROC curves approaching the ideal top-left region and AUC values above 0.97. However, MobileNetV2 achieves a higher AUC (0.9875 vs. 0.9750), indicating consistently better class separation. Although the improvement is modest, it is meaningful in high-performance settings, particularly as MobileNetV2 demonstrates superior behavior at low false positive rates, making it more reliable for safety-critical PPE detection tasks.

Precision-Recall Analysis

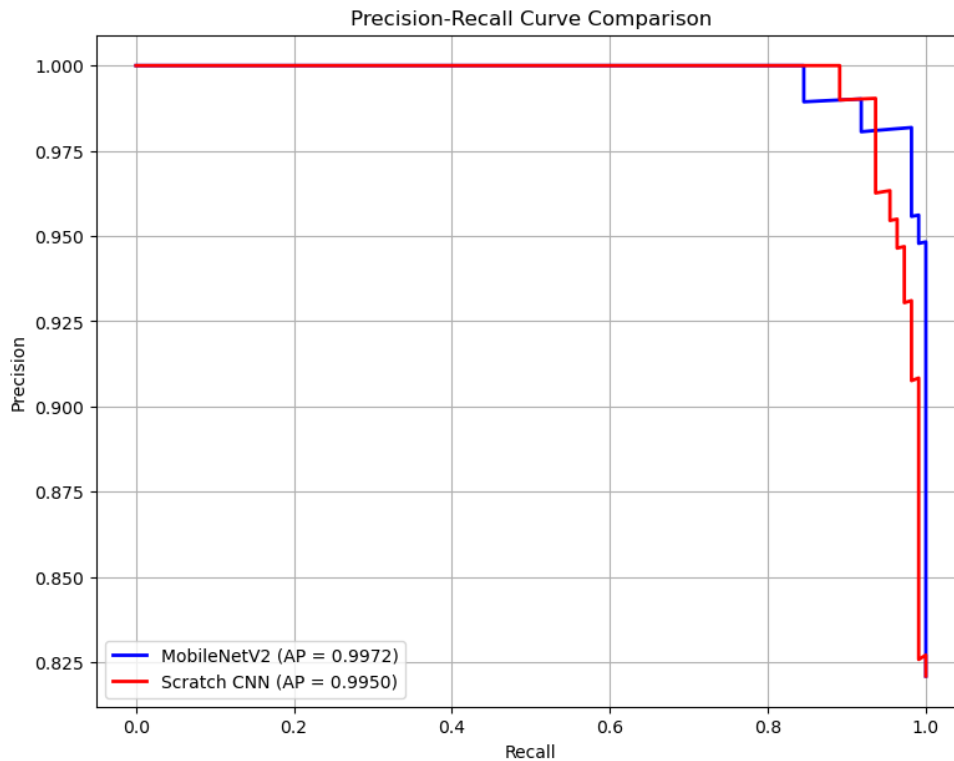


Figure 9: Precision-Recall Curve

Source: Authors' construct based on data from Kaggle.

The Precision - Recall analysis shows that both models perform exceptionally well, maintaining near-perfect precision across most recall levels, which indicates very few false positives even as more positive cases are identified. However, MobileNetV2 slightly outperforms the scratch CNN, achieving a higher Average Precision (0.9972 vs. 0.9950), meaning it is marginally more reliable when balancing precision and recall. Overall, the results confirm that both models are highly effective, but MobileNetV2 provides a small yet consistent advantage, especially at higher recall levels where performance typically begins to decline.

Honest Assessment of Near-Perfect AP Scores

Both models report AP scores close to 1.0. The notebook itself acknowledges: "The high PR score is common in this kind of dataset (especially after oversampling)." Because oversampling via `np.repeat` creates exact pixel duplicates in the training data, the decision

boundary is artificially easy to learn. These scores should be validated on genuinely novel, non-oversampled test images before drawing conclusions about real-world generalisation.

Error Analysis (Misclassified Examples)



Figure 10: Misclassified Examples

Source: Authors' construct based on data from Kaggle.

The misclassified examples from the MobileNetV2 model highlight two main error types: false positives, where non-helmet images are incorrectly classified as helmet due to visual similarities such as bright colors or head-like shapes, and false negatives, where actual helmets are missed due to occlusion, unusual viewing angles, or complex backgrounds. A dedicated error analysis further reveals that these misclassifications are strongly influenced by dataset and image-level characteristics. In particular, failures often occur in cases involving distant workers where helmets occupy very few pixels at the 224×224 input resolution, partially occluded helmets such as side or rear views and extreme lighting conditions including strong shadows or glare that reduce feature visibility. Additionally, potential label noise introduced during dataset filtering and class extraction

may contribute to ambiguous training signals. Overall, these errors are best understood as edge cases arising from real-world visual complexity and dataset limitations rather than fundamental model weaknesses.

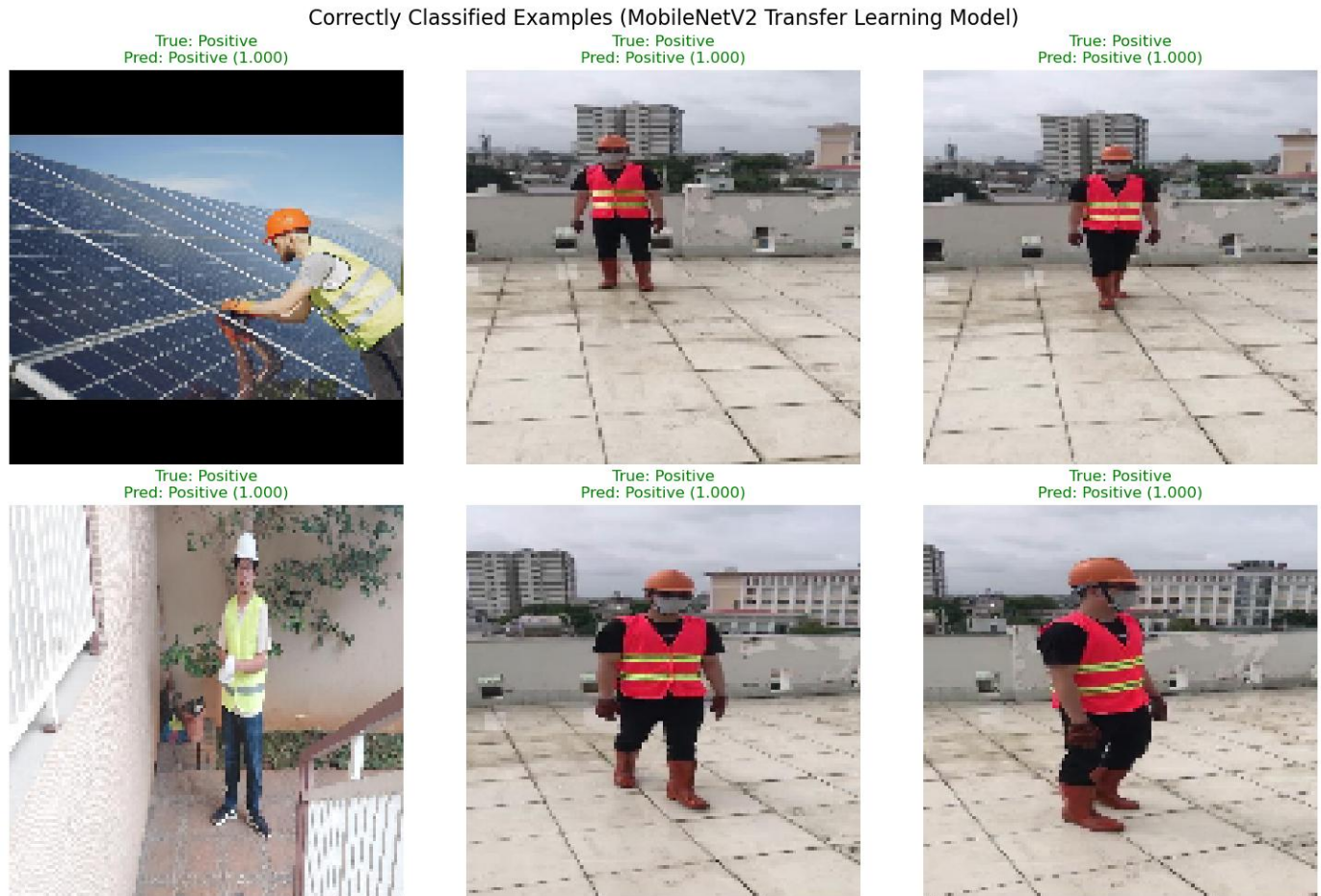


Figure 11: Correctly Classified Examples

Source: Authors' construct based on data from Kaggle.

The correctly classified examples demonstrate the strong learning capability of the MobileNetV2 model in distinguishing between helmet and no-helmet classes. In these cases, the model accurately identifies individuals wearing helmets across different positions, angles and environments, as well as correctly recognizing individuals without helmets in various contexts.

Table 11: Performance Summary

Model	Accuracy	Precision (Weighted)	Recall (Weighted)	F1-score (Weighted)	AUC (ROC)	Average Precision (PR)
Scratch CNN (Baseline)	0.90	0.89	0.90	0.89	-	-
Scratch CNN (Class Weight)	0.90	0.89	0.90	0.89	-	-
Scratch CNN (Oversampling)	0.93	0.93	0.93	0.93	0.9750	0.9950
MobileNetV2 (Transfer Learning)	0.99	0.99	0.99	0.99	0.9875	0.9972

Source: Authors' construct based on data from Kaggle.

6. HOW TO REPRODUCE

Table 12: Environment

Component	Specification
Python	3.9 or 3.10 (TensorFlow 2.x compatible)
Operating System	Windows 10 / 11 (development environment; Linux also supported)
TensorFlow	2.x — GPU support recommended; CPU-only is functional but slower
GPU (optional)	CUDA-compatible GPU — verify with <code>tf.config.list_physical_devices('GPU')</code>
Key Libraries	tensorflow, numpy, scikit-learn, matplotlib, seaborn, scikit-image, tqdm, pyyaml, h5py, streamlit, Pillow

Source: Authors' construct based on data from Kaggle.

Step-by-Step Instructions

Step 1: Download and Organise the Dataset

Download the dataset from Kaggle and arrange it in the following directory structure:

```
PPE DATASET/  
  images/  
    train/      <- training images (.jpg / .jpeg / .png)  
    val/        <- validation images  
    test/       <- test images  
  labels/  
    train/      <- YOLO .txt annotation files (one per image)  
    val/  
    test/
```

Update the `BASE_DIR` variable in the notebook if your local path differs from the default:

```
BASE_DIR = r"C:\path\to\your\PPE DATASET"
```

Step 2: Install Dependencies

Create a requirements.txt file with the following contents:

```
tensorflow>=2.10
numpy
scikit-learn
matplotlib
seaborn
scikit-image
tqdm
pyyaml
h5py
streamlit
Pillow
```

Then install all dependencies with:

```
pip install -r requirements.txt
```

Step 3: Run the Training Notebook

Open PPE_Compliance_Monitoring_FINAL.ipynb in Jupyter Notebook or JupyterLab and execute all cells from top to bottom. The notebook runs in the following sequential order:

1. Part 1: Data loading and class distribution exploration
2. Part 2: Normalisation and preprocessing
3. Part 3 (Model A): Custom CNN: baseline, class weights, oversampling
4. Transfer Learning: MobileNetV2: frozen base training, then fine-tuning
5. Comparison: Confusion matrices, ROC curves, Precision-Recall curves
6. Deployment prep: Model saved as mobilenet_ppe_model.keras and class_names.json

```
jupyter notebook PPE_Compliance_Monitoring_FINAL.ipynb
```

Step 4: Launch the Streamlit Application

After the notebook completes successfully, mobilenet_ppe_model.keras will exist in the working directory. Run the following commands to launch the app:

```
pip install --upgrade streamlit
streamlit run PPE_appNewFinal.py
```

The application will open automatically in your default browser at <http://localhost:8501>. Navigate to the Prediction page, upload a worker image and click Analyze Image.

Step 5: Reproduce the Reported Metrics

To reproduce the final test metrics without full retraining, load the saved model and run evaluation directly:

```
from tensorflow.keras.models import load_model
model = load_model("mobilenet_ppe_model.keras")
test_loss, test_acc = model.evaluate(x_test_processed, y_test)
print(f"Test Accuracy: {test_acc:.4f}")
```

Reproducibility Note

Global seeds (SEED=42) are set at the top of the notebook across Python random, NumPy and TensorFlow. Minor run-to-run variation (approximately $\pm 0.5\%$) may occur due to GPU non-determinism. Results should be consistent to within this range across machines with equivalent hardware.

Conclusion, Limitations and Recommendations

Conclusion

This study successfully developed and evaluated deep learning models for helmet detection using both a Scratch CNN (Oversampling) and a MobileNetV2 transfer learning approach. The results demonstrate that transfer learning significantly improves performance, with MobileNetV2 achieving the highest accuracy (approximately 98.5%) along with superior ROC-AUC and Precision-Recall scores. The model shows strong generalization in detecting helmets under typical conditions and proves suitable for real-world safety monitoring applications. Overall, the findings confirm that pre-trained models provide a more efficient and reliable solution compared to training from scratch, especially when working with limited datasets.

Limitations

Despite the strong performance of the model, several limitations were identified. A critical limitation is dataset bias, the model shows reduced performance when predicting images of individuals with

darker skin tones, likely due to the dataset being predominantly composed of lighter-skinned individuals, leading to poor representation and biased learning.

Additionally, the dataset lacks sufficient diversity in terms of environmental conditions, lighting variations and subject demographics, which limits the model's ability to generalize effectively.

The model is also sensitive to challenging visual conditions, such as occluded helmets, small or distant objects, and extreme lighting scenarios including shadows and glare, which can negatively impact prediction accuracy.

To end with, the model is constrained by its binary classification design, as it only distinguishes between helmet and no helmet and does not account for other types of personal protective equipment or improper helmet usage.

Recommendations

To improve the system and address the identified limitations, several recommendations are proposed. First, dataset diversity should be enhanced by collecting and including more images that represent diverse populations, particularly underrepresented groups, as well as a wider range of environments and lighting conditions to reduce bias and improve fairness.

In addition, advanced data augmentation techniques and proper class balancing should be applied to strengthen model robustness and generalization.

The system can also be improved by incorporating context-aware approaches, such as scene understanding or object detection models like YOLO, to better interpret real-world scenarios. Furthermore, exploring more advanced model architectures or ensemble methods could help improve overall accuracy and reduce errors in edge cases.

Finally, the model should be tested in real-time deployment environments, such as CCTV systems, to evaluate its practical performance and reliability in real-world applications.

Appendix: Project File Reference

File / Folder	Purpose
PPE_Compliance_Monitoring_FINAL.ipynb	Main training, evaluation, and deployment notebook
PPE_appNewFinal.py	Streamlit web application for real-time compliance checking
mobilenet_ppe_model.keras	Saved MobileNetV2 model in Keras v3 format — load for inference
class_names.json	Label mapping: ["No Helmet", "Helmet"]
requirements.txt	All Python package dependencies
PPE DATASET/images/	Training, validation, and test images by split
PPE DATASET/labels/	YOLO annotation .txt files corresponding to each image
Link to the Streamlit App	https://gvjc5clmrdk5suubyxu2bw.streamlit.app/

Source: Authors' construct based on data from Kaggle.

REFERENCES

- Thrive Plus Advanced ML & AI, *Module 3: Computer Vision – Week 7: Building Your First CV Model*, “The Scenario: Health-tech NGO chest X-ray classification using CNNs and transfer learning,” Computer Vision Module, 2026.
- Howard, A. G., et al. (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. The foundational paper for MobileNetV2 architecture, widely used for efficient transfer learning in edge/real-time applications like PPE detection.